

BACnet Protocol Implementation Conformance Statement (PICS)

Chipkin BACnet/SC Hub (B-SCHUB) example, version 1.4.0

Date: 2026-09-26. Source, manual and releases: <https://github.com/chipkin/BACnetProfileExample-B-SCHUB-CPP>.

The vendor, model and version rows below come from the CHANGE ALL OF THIS BEFORE YOU SHIP block at the top of main.cpp. If you build your own product from this example, update that block and this document. This device has **not** been submitted for BTL certification.

1. Product description

Vendor Name	Chipkin Automation Systems
Vendor Identifier	389
Product Name	CAS BACnet Stack Example - B-SCHUB
Product Model Number	CAS BACnet Stack Example - B-SCHUB
Application Software Version	1.4.0
Firmware Revision	6.0.23.0 (the CAS BACnet Stack version)
BACnet Protocol Version	1
BACnet Protocol Revision	30

Product Description: a BACnet Secure Connect hub. BACnet/SC devices connect to it over TLS 1.3 WebSockets with mutual certificate authentication, and it relays their traffic. It can also connect out to another hub as a node. A BACnet/IP port stays active for discovery and management. Its certificates can be replaced over BACnet with the BACnet/SC certificate procedures. Built on the CAS BACnet Stack.

2. BACnet standardized device profile (Annex L)

B-SCHUB - BACnet Secure Connect Hub.

This device claims exactly one profile. Because the B-SCHUB requirements are a superset of B-GENERAL's, a conformant B-SCHUB device also satisfies **B-GENERAL** (Annex L.8); that is subsumption, not a second claim.

3. BIBBs supported (Annex K)

BIBB	Description	Notes
DS-RP-B	Data Sharing - ReadProperty - B	
DS-RPM-B	Data Sharing - ReadPropertyMultiple - B	
DM-DDB-B	Device Management - Dynamic Device Binding - B	
DM-DOB-B	Device Management - Dynamic Object Binding - B	
DM-DCC-B	Device Management - DeviceCommunicationControl - B	optional password
NM-SCH-B	Network Management - BACnet/SC Hub Function - B	Network Port 2; see §9

No other BIBBs are supported. In particular this device does **not** claim DS-WP-B, DS-COV-B, any alarm and event (AE-) *BIBB*, *scheduling* (SCHED-), or trending (T-*).

It executes the device-B side of the BACnet/SC certificate procedures (135-2024 clause 19.8.3): WriteProperty File_Size and AtomicWriteFile into the operational and issuer certificate File objects, applied by ReinitializeDevice ACTIVATE_CHANGES/WARMSTART after validation, and the Network Port Command values DISCARD_CHANGES (drops staged certificate writes) and GENERATE_CSR_FILE (new key pair and certificate signing request, Network Port 2).

4. Application services supported

Service	Initiate	Execute
ReadProperty	no	yes
ReadPropertyMultiple	no	yes
Who-Is	no	yes
I-Am	yes	-
Who-Has	no	yes
I-Have	yes	-
DeviceCommunicationControl	no	yes
AtomicReadFile	no	yes
AtomicWriteFile	no	yes (certificate File objects 1, 3, 4)
WriteProperty	no	yes (File_Size of File objects 1, 3, 4, and Network Port Command, only)
ReinitializeDevice	no	yes (ACTIVATE_CHANGES, WARMSTART only)

An unsolicited I-Am is broadcast at start-up, as well as in response to Who-Is.

Any other confirmed service is rejected, and WriteProperty to any other property is refused with write-access-denied.

5. Segmentation capability

Segmentation is **supported in both directions** (Segmentation_Supported = segmented-both). Segmentation is part of the application layer, so it applies to both Network Ports.

Able to transmit segmented messages	Yes. Window size 1. No limit on the number of segments sent.
Able to receive segmented messages	Yes. Window size 1. Max_Segments_Accepted = 16.
Max_APDU_Length_Accepted	1476 octets (the BACnet/IP maximum; BACnet/SC carries the same 1476).
APDU_Segment_Timeout	5000 ms

tests/sc/segmentation_test.py checks both directions: a ReadPropertyMultiple answer segmented for a client that accepts only 128-octet APDUs, and a segmented ReadPropertyMultiple request longer than 1476 octets. It runs over BACnet/IP; no BACnet/SC client used in testing can force segmentation.

6. Standard object types supported

No object is dynamically creatable or deletable. The only writable properties are File_Size of File objects 1, 3 and 4 and Command of both Network Port objects (certificate procedures).

Object type	Instance	Object_Name	Optional properties supported
Device	389022	Chipkin Example B-SCHUB	Description
Analog Input	1	Bronze	Description
Network Port	1	BACnet IP	Description
Network Port	2	BACnet SC	Description
File	1	Operational Certificate	Description
File	2	Certificate Signing Request	Description
File	3	Issuer Certificate Slot 1	Description
File	4	Issuer Certificate Slot 2	Description

The device instance is configurable with --deviceID or the config file's device-id (BACnet requires the device instance to be configurable).

7. Data link layer options

BACnet/IP (Annex J), UDP port 47808 (0xBAC0) by default, configurable with `--port`. Active by default; `--bacnet-ip off` (config `bacnet-ip = off`) removes it, leaving a BACnet/SC-only device without Network Port 1.

BACnet/SC (Annex AB) on Network Port 2: hub function on `wss://0.0.0.0:47819/` by default (configurable with `--sc-port`), and an optional hub connector (`--sc-hub-uri`, `--sc-failover-uri`). TLS 1.3 with mutual authentication, WebSocket subprotocol `hub.bsc.bacnet.org`.

BBMD is not supported, Foreign Device registration is not supported, and MS/TP, Ethernet (Annex H) and PTP are not supported.

8. Device address binding

Static device binding is **not supported**. The device answers Who-Is/Who-Has and does not initiate any confirmed request, so it never needs to bind a peer.

9. Networking options

Not a router, not a BBMD, and does not register as a foreign device. Each Network Port reports `Network_Number 0` with `Network_Number_Quality` unknown, unless `--ip-network-number / --sc-network-number` (config `ip-network-number / sc-network-number`) sets it; it is then reported with quality configured.

The Device's `Object_Name` is configurable with `--device-name` (config `device-name`), so each installed hub can have a unique name.

BACnet/SC hub function (NM-SCH-B): accepts at most 4 connections at once (a fixed limit of this example; `--sc-max-hub-connections` can lower it) and relays unicast and broadcast BVLC-SC messages between them and the local device. Connecting devices must present a certificate that chains to an issuer in File 3 or File 4. Certificate revocation lists are not checked. The hub connector verifies the remote hub's certificate chain but not its host name (BACnet/SC certificates identify devices, not DNS names).

10. Character sets supported

UTF-8 (ANSI X3.4). Supporting a character set does not imply the device can handle data in all character sets.

11. Objects and properties

Every object this example creates, and every REQUIRED property of each (per ANSI/ASHRAE 135-2024 clause 12 and the stack's `docs/property-profile-reference.md`), plus the optional properties the example turns on. **Served by** says who answers a `ReadProperty`: the **stack** generates it, or the **app** serves it from a `GetProperty*` callback in `main.cpp`. A Δ row is a required property the app does not serve and the stack would fill with a default - that is a defect, not a feature.

Device 389022 "Chipkin Example B-SCHUB" - vendor 389 (Chipkin Automation Systems); instance configurable with --deviceId and name with --device-name. The stack rows are device-wide facts only the stack knows - the protocol version and revision it implements, the services and object types it was configured with, the live object list and address-binding table. The accepted rows are the stack's configured defaults for APDU limits, segmentation, system status and database revision; an application that answered them from its own constants could contradict the stack, so this example does not. Segmentation_Supported is segmented-both, the CAS BACnet Stack's segmentation configuration (6.x default, `cas-bacnet-stack#2992`); with it the stack also serves `Max_Segments_Accepted` (16) and `APDU_Segment_Timeout` (5000 ms) - see section 5

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
System_Status	BACnetDeviceStatus	stack default, accepted (Generic Enumerated default: 0)	no
Vendor_Name	CharacterString	app	no

Property	Datatype	Served by	Writable
Vendor_Identifier	Unsigned16	app	no
Model_Name	CharacterString	app	no
Firmware_Revision	CharacterString	app	no
Application_Software_Version	CharacterString	app	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
Protocol_Version	Unsigned	stack	no
Protocol_Revision	Unsigned	stack	no
Protocol_Services_Supported	BACnetServicesSupported	stack	no
Protocol_Object_Types_Supported	BACnetObjectTypesSupported	stack	no
Object_List	BACnetARRAY[N] of BACnetObjectIdentifier	stack	no
Max_APDU_Length_Accepted	Unsigned	stack default, accepted (CAS_BACNET_DEVICE_DEFAULT_MAX_APDU_LENGTH_ACCEPTED)	no
Segmentation_Supported	BACnetSegmentation	stack	no
APDU_Timeout	Unsigned	stack default, accepted (CAS_BACNET_DEVICE_DEFAULT_APDU_TIMEOUT)	no
Number_Of_APDU_Retries	Unsigned	stack default, accepted (CAS_BACNET_DEVICE_DEFAULT_NUMBER_OF_APDU_RETRIES)	no
Device_Address_Binding	BACnetLIST of BACnetAddressBinding	stack	no
Database_Revision	Unsigned	stack default, accepted (Generic UnsignedInteger default: 0)	no
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

Analog Input 1 "Bronze" - REAL, degrees Celsius; starts at 21.5; read-only (example data, changed with the arrow keys)

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
Present_Value	Real	app	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
Status_Flags	BACnetStatusFlags	stack	no
Event_State	BACnetEventState	stack default, accepted (Generic Enumerated default: 0)	no
Out_Of_Service	Boolean	app	no
Units	BACnetEngineeringUnits	app	no
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

Network Port 1 "BACnet IP" - BACnet/IP; kept active throughout so this example stays discoverable over plain BACnet/IP regardless of the BACnet/SC transport outcome below, unless --bacnet-ip off (BACnet/SC only) leaves this object out altogether. Network_Type and Protocol_Level are set from BACnetStack_AddNetworkPortObject()'s arguments (IPv4, BACnet Application) at start-up, not a GetProperty callback like the object's other app-served rows; Changes_Pending is

likewise computed and answered natively by the stack's Network Port object. Reliability has no fault condition this example detects, so it is accepted at the generic default (normal). Network_Number is 0 with Network_Number_Quality unknown unless --ip-network-number sets it (then quality configured); the device is not a router. Command (cl. 12.56.16) is present and writable, executed by the stack: DISCARD_CHANGES reverts a pending Network_Number

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
Status_Flags	BACnetStatusFlags	stack	no
Reliability	BACnetReliability	stack default, accepted (Generic Enumerated default: 0)	no
Out_Of_Service	Boolean	app	no
Network_Type	BACnetNetworkType	app	no
Protocol_Level	BACnetProtocolLevel	app	no
Changes_Pending	Boolean	app	no
Command (<i>optional, enabled</i>)	BACnetNetworkPortCommand	stack default (Generic Enumerated default: 0)	yes
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

Network Port 2 "BACnet SC" - BACnet/SC (Network_Type = secureConnect(11)); hosts the NM-SCH-B hub function (listener) and the optional hub connector; the WebSocket/TLS transport is sc_transport/ScTransport. Network_Type/Protocol_Level are set from BACnetStack_AddNetworkPortObject()'s arguments at start-up, same as Network Port 1. Reliability, Network_Number and Network_Number_Quality as for Network Port 1. Command (cl. 12.56.16) is writable: DISCARD_CHANGES also drops staged certificate writes, and GENERATE_CSR_FILE makes a new key pair and Certificate Signing Request (File 2) - the NetworkPortCommand callback in main.cpp

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
Status_Flags	BACnetStatusFlags	stack	no
Reliability	BACnetReliability	stack default, accepted (Generic Enumerated default: 0)	no
Out_Of_Service	Boolean	app	no
Network_Type	BACnetNetworkType	app	no
Protocol_Level	BACnetProtocolLevel	app	no
Changes_Pending	Boolean	app	no
Command (<i>optional, enabled</i>)	BACnetNetworkPortCommand	stack default (Generic Enumerated default: 0)	yes
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

File 1 "Operational Certificate" - writable over BACnet (clause 19.8.3): File_Size and AtomicWriteFile, staged until ReinitializeDevice ACTIVATE_CHANGES/WARMSTART, which validates the set (parses, matches the hub's private key, chains to an issuer) before writing operational-certificate.pem and reloading TLS. Serves the hub's operational certificate via AtomicReadFile (stream access) - bound to Network Port 2's Operational_Certificate_File. File_Size/Modification_Date come from the staged copy or the file on disk

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
File_Type	CharacterString	app	no
File_Size	Unsigned	app	yes
Modification_Date	BACnetDateTime	app	no
Archive	Boolean	app	no
Read_Only	Boolean	app	no
File_Access_Method	BACnetFileAccessMethod	stack	no
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

File 2 "Certificate Signing Request" - read-only; serves the hub's certificate signing request (certificate-signing-request.pem) - bound to Network Port 2's Certificate_Signing_Request_File. rewritten by the hub on Network Port 2 Command GENERATE_CSR_FILE, for a new key that replaces the hub's key when a certificate signed for it is activated

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
File_Type	CharacterString	app	no
File_Size	Unsigned	app	no
Modification_Date	BACnetDateTime	app	no
Archive	Boolean	app	no
Read_Only	Boolean	app	no
File_Access_Method	BACnetFileAccessMethod	stack	no
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

File 3 "Issuer Certificate Slot 1" - writable over BACnet (clause 19.8.3), same staging as File 1; issuer certificate slot 1 (issuer-certificate.pem) - one of Network Port 2's 2 Issuer_Certificate_Files entries

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
File_Type	CharacterString	app	no
File_Size	Unsigned	app	yes

Property	Datatype	Served by	Writable
Modification_Date	BACnetDateTime	app	no
Archive	Boolean	app	no
Read_Only	Boolean	app	no
File_Access_Method	BACnetFileAccessMethod	stack	no
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

File 4 "Issuer Certificate Slot 2" - writable over BACnet (clause 19.8.3), same staging as File 1; issuer certificate slot 2 (issuer-certificate-2.pem once written; until then it serves slot 1's certificate). TLS trusts every issuer in both slots (trusted-issuers.pem)

Property	Datatype	Served by	Writable
Object_Identifier	BACnetObjectIdentifier	stack	no
Object_Name	CharacterString	app	no
Object_Type	BACnetObjectType	stack	no
Description (<i>optional, enabled</i>)	CharacterString	app	no
File_Type	CharacterString	app	no
File_Size	Unsigned	app	yes
Modification_Date	BACnetDateTime	app	no
Archive	Boolean	app	no
Read_Only	Boolean	app	no
File_Access_Method	BACnetFileAccessMethod	stack	no
Property_List	BACnetARRAY[N] of BACnetPropertyIdentifier	stack	no

12. References

- ANSI/ASHRAE Standard 135-2024, Annex A (PICS template), Annex K (BIBBs), Annex L (device profiles), Annex AB (BACnet/SC), Clause 12 (object types).
- [README.md](#) - the manual.
- [TUTORIAL.md](#) - how to extend it, and how to keep this document honest when you do.